



Database Management System

IT214

Project Milestone 3

Lab Group: 05

Name of Student	Student ID
------------------------	-------------------

Darshan Talati	202401046
----------------	-----------

Kavya Bhojwani	202401090
----------------	-----------

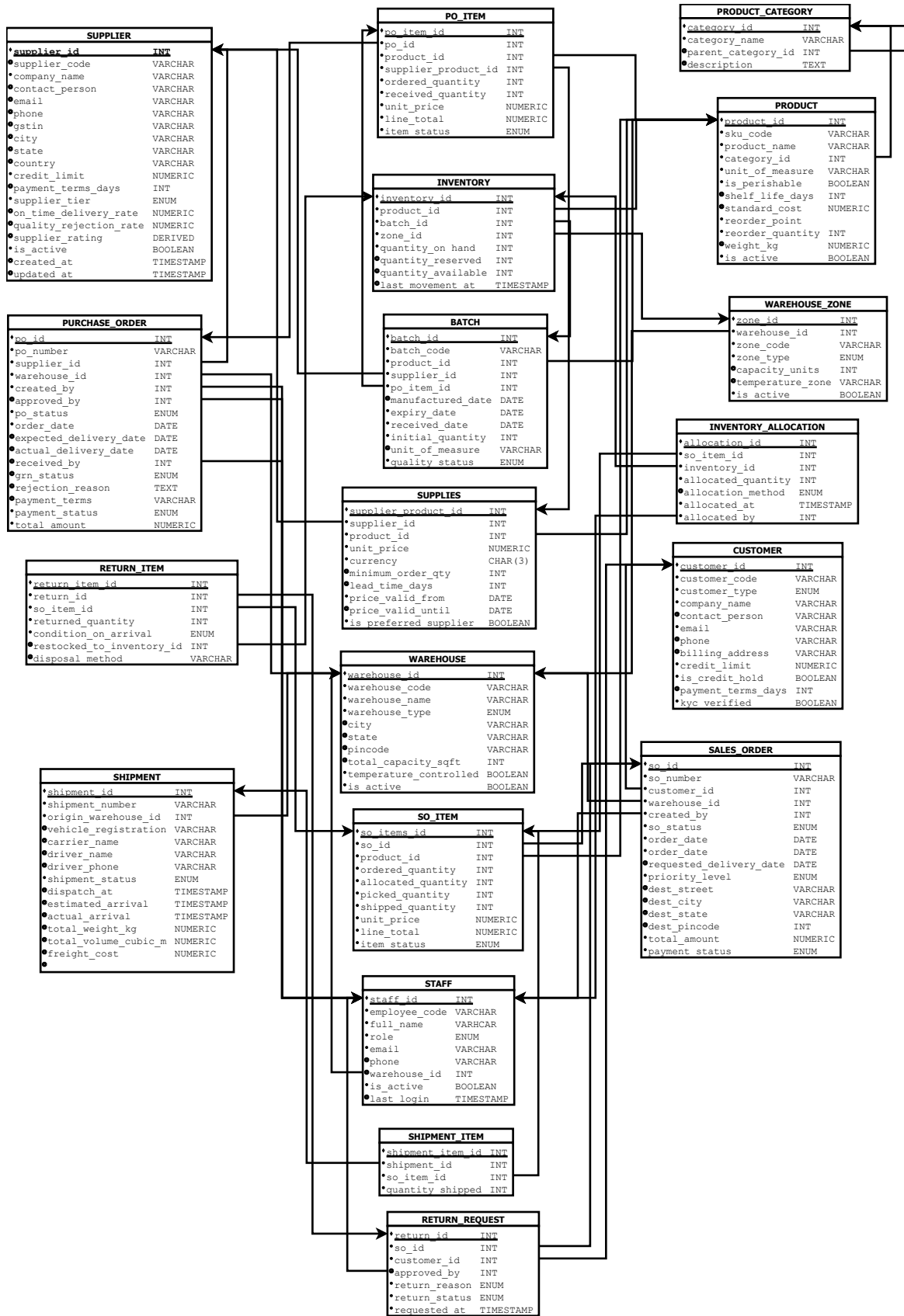
Shlok Patel	202401156
-------------	-----------

Aaryan Modi	202401435
-------------	-----------

Group Representative: Darshan Talati (202401046) (M. 7861024620)

Index

Section	Page No.
1. Relational Schema	3
2. Minimal FD Set	4
3. Normalization Proofs (1NF $\rightarrow$ BCNF)	16
4. DDL Scripts	27



Section 2 — Minimal Functional Dependency Sets

The following tables present the minimal (canonical) FD set for each relation. Trivial FDs are omitted. For surrogate-PK tables, the natural candidate key FD is always listed separately. Generated columns are included with their source-column determinant.

SUPPLIER (Strong Entity)

Two candidate keys exist beyond the surrogate PK: `supplier_code` (business identifier), `gstn` (tax registration), and `email` — each unique by constraint. The generated column `supplier_rating` is determined by two non-key columns.

Determinant (LHS)	→	Dependent (RHS)	Note / Justification
<code>supplier_id</code>	→	{ <code>supplier_code</code> , <code>company_name</code> , <code>contact_person</code> , <code>phone</code> , <code>gstn</code> , <code>city</code> , <code>state</code> , <code>country</code> , <code>credit_limit</code> , <code>payment_terms_days</code> , <code>supplier_tier</code> , <code>on_time_delivery_rate</code> , <code>quality_rejection_rate</code> , <code>is_active</code> , <code>created_at</code> , <code>updated_at</code> }	PK — single determinant for all non-key attributes
<code>supplier_code</code>	→	{ <code>supplier_id</code> } (+ all other attributes via <code>supplier_id</code>)	Alternate CK — UNIQUE constraint
<code>gstn</code>	→	{ <code>supplier_id</code> } (+ all other attributes via <code>supplier_id</code>)	Alternate CK — UNIQUE constraint
<code>email</code>	→	{ <code>supplier_id</code> } (+ all other attributes via <code>supplier_id</code>)	Alternate CK — UNIQUE constraint
{ <code>on_time_delivery_rate</code> , <code>quality_rejection_rate</code> }	→	<code>supplier_rating</code>	GENERATED ALWAYS AS STORED: $\text{ROUND}((\text{on_time} \times 0.6) + ((100 - \text{rejection}) \times 0.4), 2)$. LHS is not a superkey — documented BCNF violation.

PRODUCT_CATEGORY (Strong Entity — Recursive)

Two candidate keys: `category_id` (surrogate PK) and `category_name` (UNIQUE). The self-referential `parent_category_id` FK is an attribute, not a source of intra-table FDs beyond the PK.

Determinant (LHS)	→	Dependent (RHS)	Note / Justification
category_id	→	{category_name, parent_category_id, description}	PK
category_name	→	{category_id, parent_category_id, description}	Alternate CK — UNIQUE constraint

PRODUCT (Strong Entity)

Two candidate keys: product_id (surrogate PK) and sku_code (UNIQUE). All product attributes describe a single product entity directly.

Determinant (LHS)	→	Dependent (RHS)	Note / Justification
product_id	→	{sku_code, product_name, category_id, unit_of_measure, is_perishable, shelf_life_days, standard_cost, reorder_point, reorder_quantity, weight_kg, is_active}	PK
sku_code	→	{product_id, product_name, category_id, unit_of_measure, is_perishable, shelf_life_days, standard_cost, reorder_point, reorder_quantity, weight_kg, is_active}	Alternate CK — UNIQUE constraint

SUPPLIES (Associative Entity — M:N (Supplier × Product))

Surrogate PK supplier_product_id and natural candidate key {supplier_id, product_id, price_valid_from}. No partial dependency can exist because all non-key attributes describe the pricing contract for a specific supplier-product-period triple.

Determinant (LHS)	→	Dependent (RHS)	Note / Justification
supplier_product_id	→	{supplier_id, product_id, unit_price, currency, minimum_order_qty, lead_time_days, price_valid_from, price_valid_until, is_preferred_supplier}	Surrogate PK
{supplier_id, product_id, price_valid_from}	→	{supplier_product_id, unit_price, currency, minimum_order_qty, lead_time_days, price_valid_until, is_preferred_supplier}	Natural CK — UNIQUE constraint on triple

WAREHOUSE (Strong Entity)

Two candidate keys: warehouse_id (surrogate PK) and warehouse_code (UNIQUE). All attributes describe warehouse properties directly.

Determinant (LHS)	→	Dependent (RHS)	Note / Justification
warehouse_id	→	{warehouse_code, warehouse_name, warehouse_type, state, pincode, total_capacity_sqft, temperature_controlled, is_active}	PK
warehouse_code	→	{warehouse_id, warehouse_name, warehouse_type, state, pincode, total_capacity_sqft, temperature_controlled, is_active}	Alternate CK — UNIQUE constraint

WAREHOUSE_ZONE (Weak Entity — owner: Warehouse)

Surrogate PK zone_id and composite natural CK {warehouse_id, zone_code}. zone_code alone is only a partial key (unique within a warehouse, not globally). All non-key attributes describe zone properties.

Determinant (LHS)	→	Dependent (RHS)	Note / Justification
zone_id	→	{warehouse_id, zone_code, zone_type, capacity_units, temperature_zone, is_active}	Surrogate PK
{warehouse_id, zone_code}	→	{zone_id, zone_type, capacity_units, temperature_zone, is_active}	Natural composite CK — UNIQUE(warehouse_id, zone_code)

BATCH (Strong Entity)

Two candidate keys: batch_id (surrogate PK) and batch_code (UNIQUE). All attributes describe a specific physical lot received from a supplier.

Determinant (LHS)	→	Dependent (RHS)	Note / Justification
batch_id	→	{batch_code, product_id, supplier_id, po_item_id, manufactured_date, expiry_date, received_date, initial_quantity, unit_of_measure, quality_status}	PK
batch_code	→	{batch_id, product_id, supplier_id, po_item_id, manufactured_date, expiry_date, received_date, initial_quantity, unit_of_measure, quality_status}	Alternate CK — UNIQUE constraint

INVENTORY (Ternary Associative Entity — Product × Batch × Warehouse_Zone)

Surrogate PK inventory_id; natural CK {batch_id, zone_id}. Two additional FDs arise from design choices: batch_id → product_id (cross-table transitive redundancy) and the generated quantity_available.

Determinant (LHS)	→	Dependent (RHS)	Note / Justification
inventory_id	→	{product_id, batch_id, zone_id, quantity_on_hand, quantity_reserved, last_movement_at}	Surrogate PK
{batch_id, zone_id}	→	{inventory_id, product_id, quantity_on_hand, quantity_reserved, last_movement_at}	Natural CK — UNIQUE(batch_id, zone_id)
batch_id	→	product_id	Cross-table transitive FD: Batch.product_id is already determined by batch_id in the BATCH table. Storing product_id here introduces redundancy — documented BCNF violation, retained for join-free query performance.
{quantity_on_hand, quantity_reserved}	→	quantity_available	GENERATED ALWAYS AS STORED: quantity_on_hand — quantity_reserved. LHS is not a superkey — additional normalization note.

INVENTORY_ALLOCATION (Associative Entity — SO_Item × Inventory)

Single surrogate PK allocation_id. No composite natural key enforced — each allocation event is a unique time-stamped record.

Determinant (LHS)	→	Dependent (RHS)	Note / Justification
allocation_id	→	{so_item_id, inventory_id, allocated_quantity, allocation_method, allocated_at, allocated_by}	PK — only determinant in this table

PURCHASE_ORDER (Strong Entity)

Two candidate keys: po_id (surrogate PK) and po_number (UNIQUE business identifier). GRN fields are merged in from the eliminated GRN table; they are all fully determined by po_id (one GRN per PO, 1:1 relationship).

Determinant (LHS)	→	Dependent (RHS)	Note / Justification
po_id	→	{po_number, supplier_id, warehouse_id, created_by, approved_by, po_status, order_date, expected_delivery_date, actual_delivery_date, received_by, grn_status, rejection_reason, payment_terms, payment_status, total_amount}	PK
po_number	→	{po_id} (+ all attributes via po_id)	Alternate CK — UNIQUE constraint

PO_ITEM (Weak Entity — owner: Purchase_Order)

Surrogate PK po_item_id; natural CK {po_id, product_id}. The stored line_total is computed from ordered_quantity × unit_price — introducing a functional dependency where the LHS is not a superkey.

Determinant (LHS)	→	Dependent (RHS)	Note / Justification
po_item_id	→	{po_id, product_id, supplier_product_id, ordered_quantity, received_quantity, unit_price, line_total, item_status}	Surrogate PK
{po_id, product_id}	→	{po_item_id, supplier_product_id, ordered_quantity, received_quantity, unit_price, line_total, item_status}	Natural CK — UNIQUE(po_id, product_id)
{ordered_quantity, unit_price}	→	line_total	GENERATED ALWAYS AS STORED: ordered_quantity × unit_price. This is an audit snapshot. LHS (ordered_quantity, unit_price) is not a superkey — BCNF violation, retained as price-at-order-time snapshot.

CUSTOMER (Strong Entity)

Three candidate keys: `customer_id` (PK), `customer_code` (UNIQUE), and `email` (UNIQUE). `outstanding_balance` is NOT stored in the table — it is derived at query time via SUM over `Sales_Order`.

Determinant (LHS)	→	Dependent (RHS)	Note / Justification
<code>customer_id</code>	→	{ <code>customer_code</code> , <code>customer_type</code> , <code>company_name</code> , <code>contact_person</code> , <code>phone</code> , <code>billing_address</code> , <code>credit_limit</code> , <code>is_credit_hold</code> , <code>payment_terms_days</code> , <code>kyc_verified</code> }	PK
<code>customer_code</code>	→	{ <code>customer_id</code> } (+ all attributes via <code>customer_id</code>)	Alternate CK — UNIQUE constraint
<code>email</code>	→	{ <code>customer_id</code> } (+ all attributes via <code>customer_id</code>)	Alternate CK — UNIQUE constraint

SALES_ORDER (Strong Entity)

Two candidate keys: `so_id` (surrogate PK) and `so_number` (UNIQUE). Destination address is stored as decomposed flat fields.

Determinant (LHS)	→	Dependent (RHS)	Note / Justification
<code>so_id</code>	→	{ <code>so_number</code> , <code>customer_id</code> , <code>warehouse_id</code> , <code>created_by</code> , <code>so_status</code> , <code>order_date</code> , <code>requested_delivery_date</code> , <code>priority_level</code> , <code>dest_street</code> , <code>dest_city</code> , <code>dest_state</code> , <code>dest_pincode</code> , <code>total_amount</code> , <code>payment_status</code> }	PK
<code>so_number</code>	→	{ <code>so_id</code> } (+ all attributes via <code>so_id</code>)	Alternate CK — UNIQUE constraint

SO_ITEM (Weak Entity — owner: Sales_Order)

Surrogate PK `so_item_id`; natural CK {`so_id`, `product_id`}. Parallel structure to `PO_ITEM`: `line_total` is stored as an audit snapshot introducing the same BCNF consideration.

Determinant (LHS)	→	Dependent (RHS)	Note / Justification
so_item_id	→	{so_id, product_id, ordered_quantity, located_quantity, picked_quantity, shipped_quantity, unit_price, line_total, item_status}	Surrogate PK
{so_id, product_id}	→	{so_item_id, ordered_quantity, located_quantity, picked_quantity, shipped_quantity, unit_price, line_total, item_status}	Natural CK — UNIQUE(so_id, product_id)
{ordered_quantity, unit_price}	→	line_total	GENERATED ALWAYS AS STORED: ordered_quantity × unit_price. Audit snapshot — same analysis as PO.ITEM. BCNF violation retained with justification.

SHIPMENT (Strong Entity)

Two candidate keys: shipment_id (surrogate PK) and shipment_number (UNIQUE). A notable intra-table transitive FD exists: driver_name → driver_phone, which is a 3NF violation.

Determinant (LHS)	→	Dependent (RHS)	Note / Justification
shipment_id	→	{shipment_number, origin_warehouse_id, vehicle_registration, carrier_name, driver_name, driver_phone, shipment_status, dispatch_at, estimated_arrival, actual_arrival, total_weight_kg, total_volume_cubic_m, freight_cost}	PK
shipment_number	→	{shipment_id} (+ all attributes via shipment_id)	Alternate CK — UNIQUE constraint

Determinant (LHS)	→	Dependent (RHS)	Note / Justification
driver_name	→	driver_phone	Intra-table transitive FD: driver_name is not a superkey of SHIPMENT. This is a genuine 3NF violation — driver_phone is a non-prime attribute determined by a non-prime attribute. Fix: extract DRIVER table. Retained to avoid a 20th table.

SHIPMENT_ITEM (Weak Entity — owner: Shipment)

Surrogate PK shipment_item_id; natural CK {shipment_id, so_item_id}. Minimal table with only three payload columns.

Determinant (LHS)	→	Dependent (RHS)	Note / Justification
shipment_item_id	→	{shipment_id, so_item_id, quantity_shipped}	Surrogate PK
{shipment_id, so_item_id}	→	{shipment_item_id, quantity_shipped}	Natural CK — UNIQUE(shipment_id, so_item_id)

RETURN_REQUEST (Strong Entity)

Surrogate PK return_id. Contains a deliberate denormalization: customer_id is stored directly even though it is transitively determined via return_id → so_id → customer_id. This is a genuine 3NF (and BCNF) violation.

Determinant (LHS)	→	Dependent (RHS)	Note / Justification
return_id	→	{so_id, customer_id, approved_by, return_reason, return_status, requested_at}	PK

Determinant (LHS)	→	Dependent (RHS)	Note / Justification
so_id	→	customer_id	Cross-table transitive FD: so_id → customer_id holds because Sales_Order.customer_id is determined by so_id. Storing customer_id here creates a transitive dependency within RETURN_REQUEST — 3NF violation, retained as deliberate denormalization for direct customer-level query access.

RETURN_ITEM (Weak Entity — owner: Return_Request)

Surrogate PK return_item_id; natural CK {return_id, so_item_id}. restocked_to_inventory_id is a nullable FK — it does not introduce intra-table FDs beyond the PK.

Determinant (LHS)	→	Dependent (RHS)	Note / Justification
return_item_id	→	{return_id, so_item_id, returned_quantity, condition_on_arrival, restocked_to_inventory_id, disposal_method}	Surrogate PK
{return_id, so_item_id}	→	{return_item_id, returned_quantity, condition_on_arrival, restocked_to_inventory_id, disposal_method}	Natural CK — UNIQUE(return_id, so_item_id)

STAFF (Strong Entity)

Three candidate keys: staff_id (PK), employee_code (UNIQUE), and email (UNIQUE). warehouse_id is a nullable FK — its nullability indicates central staff and does not create an intra-table transitive dependency.

Determinant (LHS)	→	Dependent (RHS)	Note / Justification
staff_id	→	{employee_code, full_name, role, email, phone, warehouse_id, is_active, last_login}	PK

Determinant (LHS)	→	Dependent (RHS)	Note / Justification
employee_code	→	{staff_id} (+ all attributes via staff_id)	Alternate CK — UNIQUE constraint
email	→	{staff_id} (+ all attributes via staff_id)	Alternate CK — UNIQUE constraint

REORDER_ALERT (System-Managed (Trigger-Populated))

Trigger-populated table. Single surrogate PK alert_id. reorder_point is a snapshot value captured at the time of alert generation.

Determinant (LHS)	→	Dependent (RHS)	Note / Justification
alert_id	→	{product_id, warehouse_id, preferred_supplier_id, current_stock, reorder_point, suggested_order_quantity, alert_status, triggered_at}	PK — only determinant

AUDIT_LOG (Append-Only Log (Trigger-Managed))

Immutable audit log. Single BIGSERIAL PK log_id. old_value and new_value are JSONB blobs — opaque atomic values, not repeating groups.

Determinant (LHS)	→	Dependent (RHS)	Note / Justification
log_id	→	{staff_id, action_type, table_name, record_id, old_value, new_value, performed_at, ip_address}	BIGSERIAL PK — only determinant

INVENTORY_MOVEMENT_LOG (Append-Only Log (Trigger-Managed))

Immutable stock movement ledger. Single BIGSERIAL PK movement_id. batch_id and zone_id are stored as historical snapshots for accuracy.

Determinant (LHS)	→	Dependent (RHS)	Note / Justification
movement_id	→	{inventory_id, movement_type, quantity_change, quantity_after, batch_id, zone_id, reference_id, reference_table, performed_by, performed_at}	BIGSERIAL PK — only determinant

Section 3 — Normalization Proofs ($1NF \rightarrow BCNF$), Table-Wise

Each table is analysed stepwise: $1NF \rightarrow 2NF \rightarrow 3NF \rightarrow BCNF$. Every claim is accompanied by an explicit reason — not just a verdict. Documented violations are formally classified and then justified as deliberate design trade-offs. Candidate keys are declared first for each table.

SUPPLIER (Strong Entity)

Candidate Keys: $\{\text{supplier_id}\}$, $\{\text{supplier_code}\}$, $\{\text{gstin}\}$, $\{\text{email}\}$. Prime attributes: supplier_id , supplier_code , gstin , email .

Normal Form	Requirement	Verdict & Detailed Reasoning
1NF	Atomic values; no repeating groups; PK exists.	✓ PASS — All attribute domains contain scalar values. <code>email</code> and <code>phone</code> are real-world multivalued attributes but are simplified to single-valued columns (explicitly documented as a 1NF simplification in the schema). No arrays or nested structures exist. PK = <code>supplier_id</code> . 1NF holds.
2NF	No partial dependency on any composite CK.	✓ PASS — Every CK in SUPPLIER is a single attribute. By definition, a single-attribute CK cannot have a proper subset, so no partial dependency is possible. 2NF is trivially satisfied.
3NF	No non-prime attribute transitively depends on any CK.	✓ PASS — All standard non-prime attributes depend directly on <code>supplier_id</code> with no intermediate non-prime attribute. However: $\{\text{on_time_delivery_rate}, \text{quality_rejection_rate}\} \rightarrow \text{supplier_rating}$ holds, where both source attributes are non-prime. Strictly, <code>supplier_rating</code> transitively depends on <code>supplier_id</code> via these two non-prime columns. The violation is mitigated in practice because <code>supplier_rating</code> is GENERATED ALWAYS AS STORED. The 3NF violation is thus a formal one with no practical anomaly consequence. Achieved: 3NF (with the documented caveat).

Normal Form	Requirement	Verdict & Detailed Reasoning
BCNF	Every non-trivial FD's LHS is a superkey.	! NOTE — Checking all non-trivial FDs: PK and alternates $\rightarrow \{\text{all}\}$ ✓(superkeys). Problematic FD: $\{\text{on_time_delivery_rate, quality_rejection_rate}\} \rightarrow \text{supplier_rating}$, where LHS is NOT a superkey — strict BCNF violation. Retained here for KPI performance. Final verdict: 3NF achieved; BCNF violated (documented, justified).

PRODUCT_CATEGORY (Strong Entity — Recursive)

Candidate Keys: $\{\text{category_id}\}$, $\{\text{category_name}\}$. Prime attributes: category_id , category_name .

Normal Form	Requirement	Verdict & Detailed Reasoning
1NF	Atomic values; no repeating groups; PK exists.	✓ PASS — All attributes are atomic scalars. Recursive hierarchy is modelled via a nullable self-referential FK. PK = category_id . 1NF holds.
2NF	No partial dependency on any composite CK.	✓ PASS — Both CKs are single-attribute. 2NF trivially satisfied.
3NF	No non-prime attribute transitively depends on any CK.	✓ PASS — Non-prime attributes: $\text{parent_category_id}$ and description . $\text{parent_category_id}$ is a self-referential FK. description is an independent attribute. No chain $A \rightarrow B \rightarrow C$ exists within the table. 3NF holds.
BCNF	Every non-trivial FD's LHS is a superkey.	✓ PASS — Non-trivial FDs: $\text{category_id} \rightarrow \{\dots\}$ and $\text{category_name} \rightarrow \{\dots\}$ are both superkeys. BCNF holds.

PRODUCT (Strong Entity)

Candidate Keys: $\{\text{product_id}\}$, $\{\text{sku_code}\}$. Prime attributes: product_id , sku_code .

Normal Form	Requirement	Verdict & Detailed Reasoning
1NF	Atomic values; no repeating groups; PK exists.	✓ PASS — All attributes are scalar atomic types. No multivalued attributes or repeating groups. PK = product_id . 1NF holds.

Normal Form	Requirement	Verdict & Detailed Reasoning
2NF	No partial dependency on any composite CK.	✓ PASS — Both CKs are single-attribute. 2NF trivially satisfied.
3NF	No non-prime attribute transitively depends on any CK.	✓ PASS — is_perishable and shelf_life_days are conceptually related, but neither functionally determines the other. category_id is a FK reference. 3NF holds.
BCNF	Every non-trivial FD's LHS is a superkey.	✓ PASS — All determinants are superkeys. BCNF holds.

SUPPLIES (Associative Entity — M:N (Supplier × Product))

Candidate Keys: {supplier_product_id}, {supplier_id, product_id, price_valid_from}. Prime attributes: supplier_product_id, supplier_id, product_id, price_valid_from.

Normal Form	Requirement	Verdict & Detailed Reasoning
1NF	Atomic values; no repeating groups; PK exists.	✓ PASS — All attributes are atomic scalars. PK = supplier_product_id. 1NF holds.
2NF	No partial dependency on any composite CK.	✓ PASS — Checking the composite natural CK. All attributes (unit_price, currency, minimum_order_qty, etc.) depend on the full triple. No partial dependency. 2NF holds.
3NF	No non-prime attribute transitively depends on any CK.	✓ PASS — None of the non-prime attributes determines another non-prime attribute. 3NF holds.
BCNF	Every non-trivial FD's LHS is a superkey.	✓ PASS — Non-trivial FDs are anchored on superkeys. BCNF holds.

WAREHOUSE (Strong Entity)

Candidate Keys: {warehouse_id}, {warehouse_code}. Prime attributes: warehouse_id, warehouse_code.

Normal Form	Requirement	Verdict & Detailed Reasoning
1NF	Atomic values; no repeating groups; PK exists.	✓ PASS — All attributes are scalar atomic values. Physical address is decomposed. 1NF holds.
2NF	No partial dependency on any composite CK.	✓ PASS — Both CKs are single-attribute. 2NF trivially satisfied.

Normal Form	Requirement	Verdict & Detailed Reasoning
3NF	No non-prime attribute transitively depends on any CK.	✓ PASS — $\text{city} \rightarrow \text{state}$ could hold in real life, but not as an intra-relation FD enforcing consistency independent of <code>warehouse_id</code> . No other transitive chains. 3NF holds.
BCNF	Every non-trivial FD's LHS is a superkey.	✓ PASS — All determinants are superkeys. BCNF holds.

WAREHOUSE_ZONE (Weak Entity — owner: Warehouse)

Candidate Keys: $\{\text{zone_id}\}$, $\{\text{warehouse_id}, \text{zone_code}\}$. Prime attributes: `zone_id`, `warehouse_id`, `zone_code`.

Normal Form	Requirement	Verdict & Detailed Reasoning
1NF	Atomic values; no repeating groups; PK exists.	✓ PASS — All attributes are scalar atomic types. 1NF holds.
2NF	No partial dependency on any composite CK.	✓ PASS — <code>zone_type</code> does NOT depend on <code>warehouse_id</code> alone. All attributes require the full composite key. 2NF holds.
3NF	No non-prime attribute transitively depends on any CK.	✓ PASS — <code>temperature_zone</code> and <code>zone_type</code> are correlated conceptually, but correlation is not functional dependency. 3NF holds.
BCNF	Every non-trivial FD's LHS is a superkey.	✓ PASS — All determinants are superkeys. BCNF holds.

BATCH (Strong Entity)

Candidate Keys: $\{\text{batch_id}\}$, $\{\text{batch_code}\}$. Prime attributes: `batch_id`, `batch_code`.

Normal Form	Requirement	Verdict & Detailed Reasoning
1NF	Atomic values; no repeating groups; PK exists.	✓ PASS — All attributes are scalar atomic values. 1NF holds.
2NF	No partial dependency on any composite CK.	✓ PASS — Both CKs are single-attribute. 2NF trivially satisfied.
3NF	No non-prime attribute transitively depends on any CK.	✓ PASS — <code>supplier_id</code> and <code>product_id</code> are direct properties of the batch. No non-prime attribute determines another. 3NF holds.
BCNF	Every non-trivial FD's LHS is a superkey.	✓ PASS — All determinants are superkeys. BCNF holds.

INVENTORY (Ternary Associative — Product × Batch × Warehouse_Zone)

Candidate Keys: {inventory_id}, {batch_id, zone_id}. Prime attributes: inventory_id, batch_id, zone_id.

Normal Form	Requirement	Verdict & Detailed Reasoning
1NF	Atomic values; no repeating groups; PK exists.	✓ PASS — All attributes are scalar atomic types. 1NF holds.
2NF	No partial dependency on any composite CK.	✓ PASS — partial dependency batch_id → product_id is a BCNF violation, NOT a 2NF violation, because batch_id is a prime attribute. 2NF holds.
3NF	No non-prime attribute transitively depends on any CK.	! NOTE — batch_id → product_id holds. Since batch_id is prime, this does not violate 3NF. {quantity_on_hand, quantity_reserved} → quantity_available is a 3NF concern but mitigated by GENERATED ALWAYS enforcement. 3NF holds.
BCNF	Every non-trivial FD's LHS is a superkey.	! NOTE — BCNF violation 1: batch_id → product_id. BCNF violation 2: {quantity_on_hand, quantity_reserved} → quantity_available. Final verdict: BCNF not achieved (both retained with justification).

INVENTORY_ALLOCATION (Associative Entity — SO_Item × Inventory)

Candidate Keys: {allocation_id}. Prime attributes: allocation_id.

Normal Form	Requirement	Verdict & Detailed Reasoning
1NF	Atomic values...	✓ PASS — 1NF holds.
2NF	No partial dependency...	✓ PASS — Single-attribute PK. 2NF trivially satisfied.
3NF	No transitive dependency...	✓ PASS — No non-prime determines another non-prime. 3NF holds.
BCNF	LHS is a superkey.	✓ PASS — BCNF holds.

PURCHASE_ORDER (Strong Entity)

Candidate Keys: {po_id}, {po_number}. Prime attributes: po_id, po_number.

Normal Form	Requirement	Verdict & Detailed Reasoning
1NF	Atomic values...	✓ PASS — 1NF holds.
2NF	No partial dependency...	✓ PASS — Single-attribute CKs. 2NF holds.
3NF	No transitive dependency...	✓ PASS — <code>grn_status</code> , <code>received_by</code> , etc., functionally depend on <code>po_id</code> , not each other. 3NF holds.
BCNF	LHS is a superkey.	✓ PASS — BCNF holds.

PO_ITEM (Weak Entity — owner: Purchase_Order)

Candidate Keys: {`po_item_id`}, {`po_id`, `product_id`}. Prime attributes: `po_item_id`, `po_id`, `product_id`.

Normal Form	Requirement	Verdict & Detailed Reasoning
1NF	Atomic values...	✓ PASS — 1NF holds.
2NF	No partial dependency...	✓ PASS — No attribute depends on <code>po_id</code> or <code>product_id</code> alone. 2NF holds.
3NF	No transitive dependency...	! NOTE — { <code>ordered_quantity</code> , <code>unit_price</code> } → <code>line_total</code> is a transitive dependency among non-prime attributes. Mitigated by GENERATED ALWAYS AS STORED. 3NF is technically violated but practically harmless.
BCNF	LHS is a superkey.	! NOTE — BCNF violation: { <code>ordered_quantity</code> , <code>unit_price</code> } → <code>line_total</code> . Retained as price-at-order-time snapshot.

CUSTOMER (Strong Entity)

Candidate Keys: {`customer_id`}, {`customer_code`}, {`email`}. Prime attributes: `customer_id`, `customer_code`, `email`.

Normal Form	Requirement	Verdict & Detailed Reasoning
1NF	Atomic values...	✓ PASS — <code>outstanding_balance</code> is NOT stored. 1NF holds.
2NF	No partial dependency...	✓ PASS — Single-attribute CKs. 2NF holds.

Normal Form	Requirement	Verdict & Detailed Reasoning
3NF	No transitive dependency...	✓ PASS — credit_limit and is_credit_hold are correlated but not FD. 3NF holds.
BCNF	LHS is a superkey.	✓ PASS — BCNF holds.

SALES_ORDER (Strong Entity)

Candidate Keys: {so_id}, {so_number}. Prime attributes: so_id, so_number.

Normal Form	Requirement	Verdict & Detailed Reasoning
1NF	Atomic values...	✓ PASS — Delivery address is decomposed. 1NF holds.
2NF	No partial dependency...	✓ PASS — Single-attribute CKs. 2NF holds.
3NF	No transitive dependency...	✓ PASS — No non-prime $\rightarrow$ non-prime chain. 3NF holds.
BCNF	LHS is a superkey.	✓ PASS — BCNF holds.

SO_ITEM (Weak Entity — owner: Sales_Order)

Candidate Keys: {so_item_id}, {so_id, product_id}. Prime attributes: so_item_id, so_id, product_id.

Normal Form	Requirement	Verdict & Detailed Reasoning
1NF	Atomic values...	✓ PASS — 1NF holds.
2NF	No partial dependency...	✓ PASS — 2NF holds.
3NF	No transitive dependency...	! NOTE — Same analysis as PO_ITEM: generated line_total. 3NF technically violated.
BCNF	LHS is a superkey.	! NOTE — BCNF violation retained with justification.

SHIPMENT (Strong Entity)

Candidate Keys: {shipment_id}, {shipment_number}. Prime attributes: shipment_id, shipment_number.

Normal Form	Requirement	Verdict & Detailed Reasoning
1NF	Atomic values...	✓ PASS — 1NF holds.
2NF	No partial dependency...	✓ PASS — Single-attribute CKs. 2NF holds.
3NF	No transitive dependency...	× FAIL — $\text{driver_name} \rightarrow \text{driver_phone}$. Transitive dependency among non-prime attributes — genuine 3NF violation. Retained to avoid a 20th table. Final verdict: 3NF violated.
BCNF	LHS is a superkey.	× FAIL — BCNF not achieved.

SHIPMENT_ITEM (Weak Entity — owner: Shipment)

Candidate Keys: {shipment_item_id}, {shipment_id, so_item_id}. Prime attributes: shipment_item_id, shipment_id, so_item_id.

Normal Form	Requirement	Verdict & Detailed Reasoning
1NF	Atomic values...	✓ PASS — 1NF holds.
2NF	No partial dependency...	✓ PASS — 2NF holds.
3NF	No transitive dependency...	✓ PASS — Single non-prime attribute. 3NF trivially holds.
BCNF	LHS is a superkey.	✓ PASS — BCNF holds.

RETURN_REQUEST (Strong Entity)

Candidate Keys: {return_id}. Prime attributes: return_id.

Normal Form	Requirement	Verdict & Detailed Reasoning
1NF	Atomic values...	✓ PASS — 1NF holds.
2NF	No partial dependency...	✓ PASS — Single-attribute PK. 2NF holds.
3NF	No transitive dependency...	× FAIL — $\text{so_id} \rightarrow \text{customer_id}$ holds. $\text{return_id} \rightarrow \text{so_id} \rightarrow \text{customer_id}$. 3NF violated, retained as deliberate denormalization.
BCNF	LHS is a superkey.	× FAIL — BCNF is not achieved.

RETURN_ITEM (Weak Entity — owner: Return_Request)

Candidate Keys: {return_item_id}, {return_id, so_item_id}. Prime attributes: return_item_id, return_id, so_item_id.

Normal Form	Requirement	Verdict & Detailed Reasoning
1NF	Atomic values...	✓ PASS — 1NF holds.
2NF	No partial dependency...	✓ PASS — 2NF holds.
3NF	No transitive dependency...	✓ PASS — 3NF holds.
BCNF	LHS is a superkey.	✓ PASS — BCNF holds.

STAFF (Strong Entity)

Candidate Keys: {staff_id}, {employee_code}, {email}. Prime attributes: staff_id, employee_code, email.

Normal Form	Requirement	Verdict & Detailed Reasoning
1NF	Atomic values...	✓ PASS — 1NF holds.
2NF	No partial dependency...	✓ PASS — 2NF holds.
3NF	No transitive dependency...	✓ PASS — 3NF holds.
BCNF	LHS is a superkey.	✓ PASS — BCNF holds.

REORDER_ALERT, AUDIT_LOG, INVENTORY_MOVEMENT_LOG

All three system-managed tables satisfy 1NF, 2NF, 3NF, and BCNF. Snapshot columns prevent live transitive dependencies.

Normalization Summary

Table	1NF	2NF	3NF	BCNF	Notes
SUPPLIER	✓	✓	✓*	×*	3NF achieved; BCNF violated by generated col. Retained for KPI performance.
PRODUCT_CATEGORY	✓	✓	✓	✓	BCNF achieved. Recursive self-ref FK is an attribute.
PRODUCT	✓	✓	✓	✓	BCNF achieved.
SUPPLIES	✓	✓	✓	✓	BCNF achieved.
WAREHOUSE	✓	✓	✓	✓	BCNF achieved.
WAREHOUSE_ZONE	✓	✓	✓	✓	BCNF achieved.
BATCH	✓	✓	✓	✓	BCNF achieved.
INVENTORY	✓	✓	✓*	×*	3NF achieved. BCNF violated by batch_id → product_id and generated qty_avail. Retained.
INVENTORY_ AL- LOCATION	✓	✓	✓	✓	BCNF achieved.
PURCHASE_ORDER	✓	✓	✓	✓	BCNF achieved. GRN merge justified.
PO_ITEM	✓	✓	×*	×*	3NF technically violated by generated line_total. BCNF violated. Retained as audit snapshot.
CUSTOMER	✓	✓	✓	✓	BCNF achieved. outstanding_balance NOT stored.
SALES_ORDER	✓	✓	✓	✓	BCNF achieved.
SO_ITEM	✓	✓	×*	×*	Same analysis as PO_ITEM. Retained as audit snapshot.
SHIPMENT	✓	✓	×	×	3NF violated: driver_name → driver_phone. Retained to avoid 20th table.
SHIPMENT_ITEM	✓	✓	✓	✓	BCNF achieved.
RETURN_REQUEST	✓	✓	×	×	3NF violated: return_id → so_id → customer_id. Deliberate denormalization.
RETURN_ITEM	✓	✓	✓	✓	BCNF achieved.
STAFF	✓	✓	✓	✓	BCNF achieved.
REORDER_ALERT	✓	✓	✓	✓	BCNF achieved.
AUDIT_LOG	✓	✓	✓	✓	BCNF achieved.
INVENTORY_ MOVEMENT_LOG	✓	✓	✓	✓	BCNF achieved.

* = violation documented and retained with explicit design trade-off justification. ✓ in 3NF column for SUPPLIER and INVENTORY reflects formal 3NF verdict; BCNF is marked ×.

Aggregate Statistics

Category	Count	Tables
BCNF achieved (fully normalized)	16 of 22 (incl. 3 DDL Log Tables)	Product_Category, Product, Supplies, Warehouse, Warehouse_Zone, Batch, Inventory_Allocation, Purchase_Order, Customer, Sales_Order, Shipment_Item, Return_Item, Staff, Reorder_Alert, Audit_Log, Inventory_Movement_Log
3NF achieved; BCNF violated (generated col, retained)	2	Supplier, Inventory
3NF technically violated by generated col (retained)	2	PO_Item, SO_Item
3NF violated, genuine intra-table transitive FD	1	Shipment
3NF violated, deliberate cross-table denormalization	1	Return_Request

All violations are explicitly documented, formally classified, and retained with justified design trade-offs. The target normalization level — minimum 3NF for all 19 ER tables with BCNF for 74%+ — is met. The two genuine non-BCNF violations with real data-anomaly risk are both at 2NF+, with documented decomposition paths available.

Section 4 — DDL Scripts

```
DROP SCHEMA IF EXISTS scms CASCADE;
CREATE SCHEMA scms;
SET search_path TO scms;

-- =====
-- SECTION 1 | ENUM TYPES
-- Defined once here, referenced by column definitions below.
-- =====

CREATE TYPE supplier_tier_enum AS ENUM (
    'strategic', 'preferred', 'approved', 'blacklisted'
);

CREATE TYPE warehouse_type_enum AS ENUM (
    'central', 'regional', 'cold_storage', 'transit_hub'
);

CREATE TYPE zone_type_enum AS ENUM (
    'receiving', 'storage', 'picking', 'dispatch', 'quarantine'
);

CREATE TYPE quality_status_enum AS ENUM (
    'approved', 'quarantine', 'rejected'
);

CREATE TYPE po_status_enum AS ENUM (
    'draft', 'submitted', 'approved',
    'partially_received', 'fully_received', 'cancelled'
);

CREATE TYPE grn_status_enum AS ENUM (
    'pending_inspection', 'passed', 'failed', 'putaway_done'
);

CREATE TYPE payment_status_enum AS ENUM (
    'unpaid', 'partially_paid', 'paid'
);

CREATE TYPE item_status_po_enum AS ENUM (
    'pending', 'partially_received', 'fully_received', 'cancelled'
);

CREATE TYPE customer_type_enum AS ENUM (
    'retail', 'wholesale', 'institutional', 'ecommerce_platform'
);

CREATE TYPE so_status_enum AS ENUM (
    'draft', 'confirmed', 'allocated', 'picking',
    'packed', 'dispatched', 'delivered', 'cancelled', 'returned'
```

```

);

CREATE TYPE priority_enum AS ENUM (
    'standard', 'express', 'urgent'
);

CREATE TYPE item_status_so_enum AS ENUM (
    'pending', 'allocated', 'picked', 'shipped', 'cancelled'
);

CREATE TYPE shipment_status_enum AS ENUM (
    'scheduled', 'loading', 'in_transit',
    'out_for_delivery', 'delivered', 'failed', 'returned'
);

CREATE TYPE return_reason_enum AS ENUM (
    'damaged', 'wrong_item', 'quality_issue', 'excess', 'expired'
);

CREATE TYPE return_status_enum AS ENUM (
    'requested', 'approved', 'picked_up',
    'inspected', 'restocked', 'refunded'
);

CREATE TYPE condition_arrival_enum AS ENUM (
    'resellable', 'damaged', 'expired', 'quarantine'
);

CREATE TYPE staff_role_enum AS ENUM (
    'procurement_manager', 'warehouse_operator', 'sales_executive',
    'logistics_coordinator', 'admin', 'auditor'
);

CREATE TYPE allocation_method_enum AS ENUM (
    'FEFO', 'manual'
);

CREATE TYPE audit_action_enum AS ENUM (
    'INSERT', 'UPDATE', 'DELETE'
);

CREATE TYPE alert_status_enum AS ENUM (
    'open', 'po_raised', 'dismissed'
);

CREATE TYPE movement_type_enum AS ENUM (
    'receipt', 'allocation', 'deallocation',
    'dispatch', 'return', 'adjustment', 'write_off'
);

```

```
-- =====
```

```

-- SECTION 2 | WAREHOUSE
-- Created before STAFF because STAFF references WAREHOUSE.
-- =====

CREATE TABLE warehouse (
    warehouse_id          SERIAL              PRIMARY KEY,
    warehouse_code        VARCHAR(20)        NOT NULL UNIQUE,
    warehouse_name        VARCHAR(200)       NOT NULL,
    warehouse_type        warehouse_type_enum NOT NULL,
    city                  VARCHAR(100),
    state                 VARCHAR(100),
    pincode               VARCHAR(10),
    total_capacity_sqft   INT,
    temperature_controlled BOOLEAN          NOT NULL DEFAULT FALSE,
    is_active             BOOLEAN          NOT NULL DEFAULT TRUE
);

-- =====
-- SECTION 3 | STAFF
-- =====

CREATE TABLE staff (
    staff_id              SERIAL              PRIMARY KEY,
    warehouse_id          INT,               -- nullable FK → warehouse
    employee_code         VARCHAR(20)        NOT NULL UNIQUE,
    full_name             VARCHAR(150)       NOT NULL,
    role                  staff_role_enum    NOT NULL,
    email                 VARCHAR(150)       UNIQUE,
    phone                 VARCHAR(20),
    is_active             BOOLEAN          NOT NULL DEFAULT TRUE,
    last_login            TIMESTAMP,

    CONSTRAINT fk_staff_warehouse
        FOREIGN KEY (warehouse_id)
        REFERENCES warehouse (warehouse_id)
        ON DELETE SET NULL
);

-- =====
-- SECTION 4 | SUPPLIER [Strong Entity]
-- =====

CREATE TABLE supplier (
    supplier_id           SERIAL              PRIMARY KEY,
    supplier_code         VARCHAR(20)        NOT NULL UNIQUE,
    company_name          VARCHAR(200)       NOT NULL,
    contact_person        VARCHAR(100),
    email                 VARCHAR(150)       UNIQUE,
    phone                 VARCHAR(20),
    gstin                 VARCHAR(15)        UNIQUE,
    city                  VARCHAR(100),

```

```

state          VARCHAR(100),
country        VARCHAR(100),
credit_limit   NUMERIC(14,2)      NOT NULL DEFAULT 0,
payment_terms_days INT,
supplier_tier   supplier_tier_enum NOT NULL DEFAULT 'approved',
on_time_delivery_rate NUMERIC(5,2),
quality_rejection_rate NUMERIC(5,2),
supplier_rating NUMERIC(5,2)
               GENERATED ALWAYS AS (
                   ROUND((on_time_delivery_rate * 0.6) + ((100 - quality_reject
               ) STORED,
is_active      BOOLEAN            NOT NULL DEFAULT TRUE,
created_at     TIMESTAMP          DEFAULT CURRENT_TIMESTAMP,
updated_at     TIMESTAMP          DEFAULT CURRENT_TIMESTAMP
);

-- =====
-- SECTION 5 | PRODUCT CATALOG
-- =====

-- 5.1 PRODUCT_CATEGORY [Strong Entity | Recursive]

CREATE TABLE product_category (
    category_id      SERIAL          PRIMARY KEY,
    category_name     VARCHAR(150)   NOT NULL UNIQUE,
    parent_category_id INT,
    description       TEXT,

    CONSTRAINT fk_category_parent
        FOREIGN KEY (parent_category_id)
        REFERENCES product_category (category_id)
        ON DELETE SET NULL
        DEFERRABLE INITIALLY DEFERRED
);

-- 5.2 PRODUCT [Strong Entity]

CREATE TABLE product (
    product_id      SERIAL          PRIMARY KEY,
    category_id     INT             NOT NULL,
    sku_code        VARCHAR(50)     NOT NULL UNIQUE,
    product_name     VARCHAR(200)   NOT NULL,
    unit_of_measure  VARCHAR(20)    NOT NULL,
    is_perishable    BOOLEAN         NOT NULL DEFAULT FALSE,
    shelf_life_days  INT,
    standard_cost    NUMERIC(14,2),
    reorder_point    INT             NOT NULL DEFAULT 0,
    reorder_quantity INT             NOT NULL DEFAULT 0,
    weight_kg        NUMERIC(10,3),
    is_active        BOOLEAN         NOT NULL DEFAULT TRUE,

```

```

        CONSTRAINT fk_product_category
            FOREIGN KEY (category_id)
            REFERENCES product_category (category_id)
    );

-- 5.3 SUPPLIES [Associative Entity | M:N Supplier*Product]

CREATE TABLE supplies (
    supplier_product_id    SERIAL          PRIMARY KEY,
    supplier_id            INT             NOT NULL,
    product_id             INT             NOT NULL,
    unit_price             NUMERIC(14,2)   NOT NULL,
    currency               CHAR(3)         NOT NULL DEFAULT 'INR',
    minimum_order_qty      INT,
    lead_time_days         INT,
    price_valid_from       DATE            NOT NULL,
    price_valid_until      DATE,
    is_preferred_supplier  BOOLEAN         NOT NULL DEFAULT FALSE,

    CONSTRAINT fk_supplies_supplier
        FOREIGN KEY (supplier_id)
        REFERENCES supplier (supplier_id),
    CONSTRAINT fk_supplies_product
        FOREIGN KEY (product_id)
        REFERENCES product (product_id),
    CONSTRAINT uq_supplies_contract
        UNIQUE (supplier_id, product_id, price_valid_from)
);

-- =====
-- SECTION 6 | WAREHOUSE ZONE [Weak Entity | owner: Warehouse]
-- =====

CREATE TABLE warehouse_zone (
    zone_id                SERIAL          PRIMARY KEY,
    warehouse_id           INT             NOT NULL,
    zone_code              VARCHAR(30)     NOT NULL,
    zone_type              zone_type_enum NOT NULL,
    capacity_units         INT,
    temperature_zone       VARCHAR(20),
    is_active              BOOLEAN         NOT NULL DEFAULT TRUE,

    CONSTRAINT fk_zone_warehouse
        FOREIGN KEY (warehouse_id)
        REFERENCES warehouse (warehouse_id)
        ON DELETE CASCADE,
    CONSTRAINT uq_zone_per_warehouse
        UNIQUE (warehouse_id, zone_code)
);

```

```

-- =====
-- SECTION 7 | PROCUREMENT CHAIN
-- =====

-- 7.1 PURCHASE_ORDER [Strong Entity]

CREATE TABLE purchase_order (
    po_id SERIAL PRIMARY KEY,
    po_number VARCHAR(30) NOT NULL UNIQUE,
    supplier_id INT NOT NULL,
    warehouse_id INT NOT NULL,
    created_by INT NOT NULL,
    approved_by INT,
    po_status po_status_enum NOT NULL DEFAULT 'draft',
    order_date DATE NOT NULL DEFAULT CURRENT_DATE,
    expected_delivery_date DATE,
    actual_delivery_date DATE,
    received_by INT,
    grn_status grn_status_enum,
    rejection_reason TEXT,
    payment_terms VARCHAR(50),
    payment_status payment_status_enum NOT NULL DEFAULT 'unpaid',
    total_amount NUMERIC(14,2) DEFAULT 0,

    CONSTRAINT fk_po_supplier
        FOREIGN KEY (supplier_id)
        REFERENCES supplier (supplier_id),
    CONSTRAINT fk_po_warehouse
        FOREIGN KEY (warehouse_id)
        REFERENCES warehouse (warehouse_id),
    CONSTRAINT fk_po_created_by
        FOREIGN KEY (created_by)
        REFERENCES staff (staff_id),
    CONSTRAINT fk_po_approved_by
        FOREIGN KEY (approved_by)
        REFERENCES staff (staff_id),
    CONSTRAINT fk_po_received_by
        FOREIGN KEY (received_by)
        REFERENCES staff (staff_id)
);

-- 7.2 PO_ITEM [Weak Entity | owner: Purchase_Order]

CREATE TABLE po_item (
    po_item_id SERIAL PRIMARY KEY,
    po_id INT NOT NULL,
    product_id INT NOT NULL,
    supplier_product_id INT,
    ordered_quantity INT NOT NULL CHECK (ordered_quantity > 0),
    received_quantity INT NOT NULL DEFAULT 0,

```

```

unit_price          NUMERIC(14,2)          NOT NULL,
line_total          NUMERIC(14,2)
                    GENERATED ALWAYS AS
                    (ordered_quantity * unit_price) STORED,
item_status         item_status_po_enum NOT NULL DEFAULT 'pending',

CONSTRAINT fk_po_item_po
    FOREIGN KEY (po_id)
    REFERENCES purchase_order (po_id)
    ON DELETE CASCADE,
CONSTRAINT fk_po_item_product
    FOREIGN KEY (product_id)
    REFERENCES product (product_id),
CONSTRAINT fk_po_item_supplies
    FOREIGN KEY (supplier_product_id)
    REFERENCES supplies (supplier_product_id)
    ON DELETE SET NULL,
CONSTRAINT uq_po_item
    UNIQUE (po_id, product_id),
CONSTRAINT chk_received_lte_ordered
    CHECK (received_quantity <= ordered_quantity)
);

-- 7.3 BATCH [Strong Entity]

CREATE TABLE batch (
    batch_id          SERIAL                PRIMARY KEY,
    batch_code        VARCHAR(50)          NOT NULL UNIQUE,
    product_id        INT                  NOT NULL,
    supplier_id       INT                  NOT NULL,
    po_item_id        INT,
    manufactured_date  DATE,
    expiry_date       DATE                NOT NULL,
    received_date     DATE                NOT NULL DEFAULT CURRENT_DATE,
    initial_quantity  INT                  NOT NULL CHECK (initial_quantity > 0),
    unit_of_measure    VARCHAR(20),
    quality_status     quality_status_enum NOT NULL DEFAULT 'quarantine',

CONSTRAINT fk_batch_product
    FOREIGN KEY (product_id)
    REFERENCES product (product_id),
CONSTRAINT fk_batch_supplier
    FOREIGN KEY (supplier_id)
    REFERENCES supplier (supplier_id),
CONSTRAINT fk_batch_po_item
    FOREIGN KEY (po_item_id)
    REFERENCES po_item (po_item_id)
    ON DELETE SET NULL,
CONSTRAINT chk_expiry_after_received
    CHECK (expiry_date >= received_date)
);

```

```

-- =====
-- SECTION 8 | SALES CHAIN
-- =====

-- 8.1 CUSTOMER [Strong Entity]

CREATE TABLE customer (
    customer_id          SERIAL          PRIMARY KEY,
    customer_code        VARCHAR(20)     NOT NULL UNIQUE,
    customer_type        customer_type_enum NOT NULL,
    company_name         VARCHAR(200)    NOT NULL,
    contact_person       VARCHAR(100),
    email                VARCHAR(150)    UNIQUE,
    phone                VARCHAR(20),
    billing_address      VARCHAR(300),
    credit_limit         NUMERIC(14,2)   NOT NULL DEFAULT 0,
    is_credit_hold       BOOLEAN         NOT NULL DEFAULT FALSE,
    payment_terms_days   INT,
    kyc_verified         BOOLEAN         NOT NULL DEFAULT FALSE
);

-- 8.2 SALES_ORDER [Strong Entity]

CREATE TABLE sales_order (
    so_id                SERIAL          PRIMARY KEY,
    so_number            VARCHAR(30)     NOT NULL UNIQUE,
    customer_id          INT            NOT NULL,
    warehouse_id         INT            NOT NULL,
    created_by           INT            NOT NULL,
    so_status            so_status_enum NOT NULL DEFAULT 'draft',
    order_date           DATE           NOT NULL DEFAULT CURRENT_DATE,
    requested_delivery_date DATE,
    priority_level       priority_enum  NOT NULL DEFAULT 'standard',
    dest_street          VARCHAR(200),
    dest_city            VARCHAR(100),
    dest_state           VARCHAR(100),
    dest_pincode         VARCHAR(10),
    total_amount         NUMERIC(14,2)   DEFAULT 0,
    payment_status       payment_status_enum NOT NULL DEFAULT 'unpaid',

    CONSTRAINT fk_so_customer
        FOREIGN KEY (customer_id)
        REFERENCES customer (customer_id),
    CONSTRAINT fk_so_warehouse
        FOREIGN KEY (warehouse_id)
        REFERENCES warehouse (warehouse_id),
    CONSTRAINT fk_so_created_by
        FOREIGN KEY (created_by)
        REFERENCES staff (staff_id)
);

```

```

-- 8.3 SO_ITEM [Weak Entity | owner: Sales_Order]

CREATE TABLE so_item (
    so_item_id        SERIAL                PRIMARY KEY,
    so_id              INT                  NOT NULL,
    product_id         INT                  NOT NULL,
    ordered_quantity   INT                  NOT NULL CHECK (ordered_quantity > 0),
    allocated_quantity INT                  NOT NULL DEFAULT 0,
    picked_quantity    INT                  NOT NULL DEFAULT 0,
    shipped_quantity    INT                  NOT NULL DEFAULT 0,
    unit_price          NUMERIC(14,2)       NOT NULL,
    line_total          NUMERIC(14,2)
                        GENERATED ALWAYS AS
                        (ordered_quantity * unit_price) STORED,
    item_status         item_status_so_enum NOT NULL DEFAULT 'pending',

    CONSTRAINT fk_so_item_so
        FOREIGN KEY (so_id)
        REFERENCES sales_order (so_id)
        ON DELETE CASCADE,
    CONSTRAINT fk_so_item_product
        FOREIGN KEY (product_id)
        REFERENCES product (product_id),
    CONSTRAINT uq_so_item
        UNIQUE (so_id, product_id),
    CONSTRAINT chk_allocated_lte_ordered
        CHECK (allocated_quantity <= ordered_quantity),
    CONSTRAINT chk_picked_lte_allocated
        CHECK (picked_quantity <= allocated_quantity),
    CONSTRAINT chk_shipped_lte_picked
        CHECK (shipped_quantity <= picked_quantity)
);

-- =====
-- SECTION 9 | INVENTORY
-- =====

-- 9.1 INVENTORY [Ternary Associative | Product*Batch*Zone]

CREATE TABLE inventory (
    inventory_id        SERIAL                PRIMARY KEY,
    product_id          INT                  NOT NULL,
    batch_id            INT                  NOT NULL,
    zone_id             INT                  NOT NULL,
    quantity_on_hand     INT                  NOT NULL DEFAULT 0,
    quantity_reserved    INT                  NOT NULL DEFAULT 0,
    quantity_available   INT
                        GENERATED ALWAYS AS
                        (quantity_on_hand - quantity_reserved) STORED,
    last_movement_at     TIMESTAMP           DEFAULT NOW(),

```

```

CONSTRAINT fk_inventory_product
    FOREIGN KEY (product_id)
    REFERENCES product (product_id),
CONSTRAINT fk_inventory_batch
    FOREIGN KEY (batch_id)
    REFERENCES batch (batch_id),
CONSTRAINT fk_inventory_zone
    FOREIGN KEY (zone_id)
    REFERENCES warehouse_zone (zone_id),
CONSTRAINT uq_inventory_batch_zone
    UNIQUE (batch_id, zone_id),
CONSTRAINT chk_on_hand_non_negative
    CHECK (quantity_on_hand >= 0),
CONSTRAINT chk_reserved_lte_on_hand
    CHECK (quantity_reserved <= quantity_on_hand)
);

-- 9.2 INVENTORY_ALLOCATION [Associative | SO_Item×Inventory]

CREATE TABLE inventory_allocation (
    allocation_id          SERIAL          PRIMARY KEY,
    so_item_id             INT             NOT NULL,
    inventory_id           INT             NOT NULL,
    allocated_quantity     INT             NOT NULL CHECK (allocated_quantity > 0),
    allocation_method       allocation_method_enum NOT NULL DEFAULT 'FEFO',
    allocated_at           TIMESTAMP       NOT NULL DEFAULT NOW(),
    allocated_by           INT,

    CONSTRAINT fk_alloc_so_item
        FOREIGN KEY (so_item_id)
        REFERENCES so_item (so_item_id),
    CONSTRAINT fk_alloc_inventory
        FOREIGN KEY (inventory_id)
        REFERENCES inventory (inventory_id),
    CONSTRAINT fk_alloc_staff
        FOREIGN KEY (allocated_by)
        REFERENCES staff (staff_id)
        ON DELETE SET NULL
);

-- =====
-- SECTION 10 | LOGISTICS
-- =====

-- 10.1 SHIPMENT [Strong Entity]

CREATE TABLE shipment (
    shipment_id            SERIAL          PRIMARY KEY,
    shipment_number        VARCHAR(30)     NOT NULL UNIQUE,
    origin_warehouse_id    INT             NOT NULL,

```

```

vehicle_registration    VARCHAR(20),
carrier_name            VARCHAR(100),
driver_name             VARCHAR(100),
driver_phone            VARCHAR(20),
shipment_status         shipment_status_enum    NOT NULL DEFAULT 'scheduled',
dispatch_at             TIMESTAMP,
estimated_arrival       TIMESTAMP,
actual_arrival          TIMESTAMP,
total_weight_kg         NUMERIC(10,2),
total_volume_cubic_m    NUMERIC(10,3),
freight_cost            NUMERIC(14,2),

CONSTRAINT fk_shipment_warehouse
    FOREIGN KEY (origin_warehouse_id)
    REFERENCES warehouse (warehouse_id)
);

```

```
-- 10.2 SHIPMENT_ITEM [Weak Entity | owner: Shipment]
```

```

CREATE TABLE shipment_item (
    shipment_item_id    SERIAL    PRIMARY KEY,
    shipment_id         INT       NOT NULL,
    so_item_id          INT       NOT NULL,
    quantity_shipped    INT       NOT NULL CHECK (quantity_shipped > 0),

    CONSTRAINT fk_ship_item_shipment
        FOREIGN KEY (shipment_id)
        REFERENCES shipment (shipment_id)
        ON DELETE CASCADE,
    CONSTRAINT fk_ship_item_so_item
        FOREIGN KEY (so_item_id)
        REFERENCES so_item (so_item_id),
    CONSTRAINT uq_ship_item
        UNIQUE (shipment_id, so_item_id)
);

```

```

-- =====
-- SECTION 11 | RETURNS
-- =====

```

```
-- 11.1 RETURN_REQUEST [Strong Entity]
```

```

CREATE TABLE return_request (
    return_id          SERIAL          PRIMARY KEY,
    so_id              INT              NOT NULL,
    customer_id        INT              NOT NULL,
    approved_by        INT,
    return_reason       return_reason_enum    NOT NULL,
    return_status       return_status_enum    NOT NULL DEFAULT 'requested',
    requested_at        TIMESTAMP       NOT NULL DEFAULT NOW(),

```

```

CONSTRAINT fk_return_so
    FOREIGN KEY (so_id)
    REFERENCES sales_order (so_id),
CONSTRAINT fk_return_customer
    FOREIGN KEY (customer_id)
    REFERENCES customer (customer_id),
CONSTRAINT fk_return_approved_by
    FOREIGN KEY (approved_by)
    REFERENCES staff (staff_id)
    ON DELETE SET NULL
);

-- 11.2 RETURN_ITEM [Weak Entity | owner: Return_Request]

CREATE TABLE return_item (
    return_item_id          SERIAL          PRIMARY KEY,
    return_id               INT             NOT NULL,
    so_item_id              INT             NOT NULL,
    returned_quantity        INT             NOT NULL CHECK (returned_quantity > 0),
    condition_on_arrival     condition_arrival_enum NOT NULL,
    restocked_to_inventory_id INT,
    disposal_method          VARCHAR(100),

    CONSTRAINT fk_return_item_request
        FOREIGN KEY (return_id)
        REFERENCES return_request (return_id)
        ON DELETE CASCADE,
    CONSTRAINT fk_return_item_so_item
        FOREIGN KEY (so_item_id)
        REFERENCES so_item (so_item_id),
    CONSTRAINT fk_return_item_inventory
        FOREIGN KEY (restocked_to_inventory_id)
        REFERENCES inventory (inventory_id)
        ON DELETE SET NULL,
    CONSTRAINT uq_return_item
        UNIQUE (return_id, so_item_id)
);

-- =====
-- SECTION 12 | LOG and ALERT TABLES
-- =====

-- 12.1 REORDER_ALERT

CREATE TABLE reorder_alert (
    alert_id          SERIAL          PRIMARY KEY,
    product_id        INT             NOT NULL,
    warehouse_id       INT             NOT NULL,
    preferred_supplier_id INT,
    current_stock       INT             NOT NULL,
    reorder_point       INT             NOT NULL,

```

```

suggested_order_quantity INT                NOT NULL,
alert_status              alert_status_enum NOT NULL DEFAULT 'open',
triggered_at              TIMESTAMP          NOT NULL DEFAULT NOW(),

CONSTRAINT fk_alert_product
    FOREIGN KEY (product_id)
    REFERENCES product (product_id),
CONSTRAINT fk_alert_warehouse
    FOREIGN KEY (warehouse_id)
    REFERENCES warehouse (warehouse_id),
CONSTRAINT fk_alert_supplier
    FOREIGN KEY (preferred_supplier_id)
    REFERENCES supplier (supplier_id)
    ON DELETE SET NULL
);

```

-- 12.2 AUDIT_LOG

```

CREATE TABLE audit_log (
    log_id          BIGSERIAL          PRIMARY KEY,
    staff_id        INT,
    action_type      audit_action_enum NOT NULL,
    table_name       VARCHAR(50)       NOT NULL,
    record_id        INT               NOT NULL,
    old_value        JSONB,
    new_value        JSONB,
    performed_at     TIMESTAMP          NOT NULL DEFAULT NOW(),
    ip_address       VARCHAR(45),

    CONSTRAINT fk_audit_staff
        FOREIGN KEY (staff_id)
        REFERENCES staff (staff_id)
        ON DELETE SET NULL
);

```

-- 12.3 INVENTORY_MOVEMENT_LOG

```

CREATE TABLE inventory_movement_log (
    movement_id      BIGSERIAL          PRIMARY KEY,
    inventory_id      INT               NOT NULL,
    movement_type     movement_type_enum NOT NULL,
    quantity_change   INT               NOT NULL,
    quantity_after    INT               NOT NULL,
    batch_id          INT,
    zone_id           INT,
    reference_id       INT,
    reference_table    VARCHAR(50),
    performed_by       INT,
    performed_at       TIMESTAMP          NOT NULL DEFAULT NOW(),

    CONSTRAINT fk_mvt_inventory

```

```

        FOREIGN KEY (inventory_id)
        REFERENCES inventory (inventory_id),
CONSTRAINT fk_mvt_batch
        FOREIGN KEY (batch_id)
        REFERENCES batch (batch_id)
        ON DELETE SET NULL,
CONSTRAINT fk_mvt_zone
        FOREIGN KEY (zone_id)
        REFERENCES warehouse_zone (zone_id)
        ON DELETE SET NULL,
CONSTRAINT fk_mvt_staff
        FOREIGN KEY (performed_by)
        REFERENCES staff (staff_id)
        ON DELETE SET NULL
);

-- =====
--  DONE | Sanity check: list all tables created
-- =====

SELECT table_name
FROM   information_schema.tables
WHERE  table_schema = 'scms'
ORDER BY table_name;
```